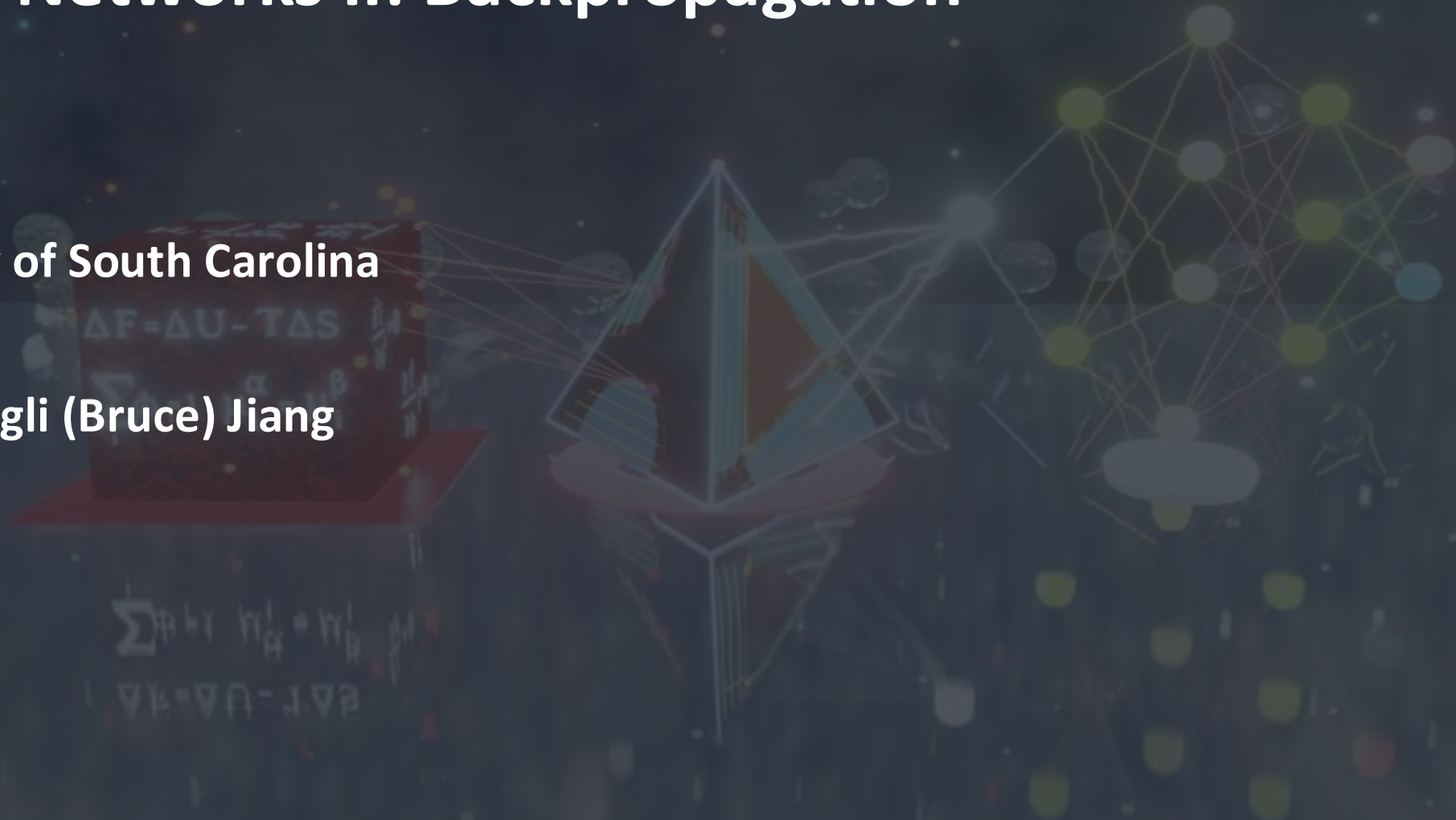


# Neural Networks II: Backpropagation

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



*From Lecture 17*

## **Neural Networks I: Introduction to NNs**

- *What a neural network (NN) is and how it is structured*
- *What activation functions do and why nonlinearity matters*
- *What TensorFlow and Keras are and how they are used for NNs*

### **In this lecture, you will learn**

- *Loss functions, back-propagation and gradient descent*
- *Train-validation-test split in neural networks*
- *Our first neural network coding practice*

# Loss Functions

A **loss function** measures how different the model prediction is from the true value.  
The choice of loss function depends on the task.

## Regression

- Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

## Classification

- Binary classification: Binary Cross-Entropy

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

- Multiclass classification: Categorical Cross-Entropy

$$\text{CCE} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{p}_{ik})$$

# Loss Function Practice

**What does  $y$  mean?**

True class:  $\mathbf{y} = [0 \quad 1 \quad 0]$

Predicted probabilities:  $\hat{\mathbf{p}} = [0.20 \quad 0.70 \quad 0.10]$

$$L = - \sum_{k=1}^3 y_k \log(\hat{p}_k)$$

**What is the categorical cross-entropy loss?**

$$L = - [0 \log(0.20) + 1 \log(0.70) + 0 \log(0.10)]$$

$$= - \log(0.70)$$

$$\approx \boxed{0.357}$$

# Backpropagation

**Backpropagation** calculates how much each weight and bias contributed to the loss.

## 1. Forward pass

- Inputs  $\rightarrow$  hidden layers  $\rightarrow$  prediction

## 2. Calculate the loss

- Compare prediction  $\hat{y}$  with true value  $y$ .

## 3. Backward pass

- Propagate the error backward and calculate gradients

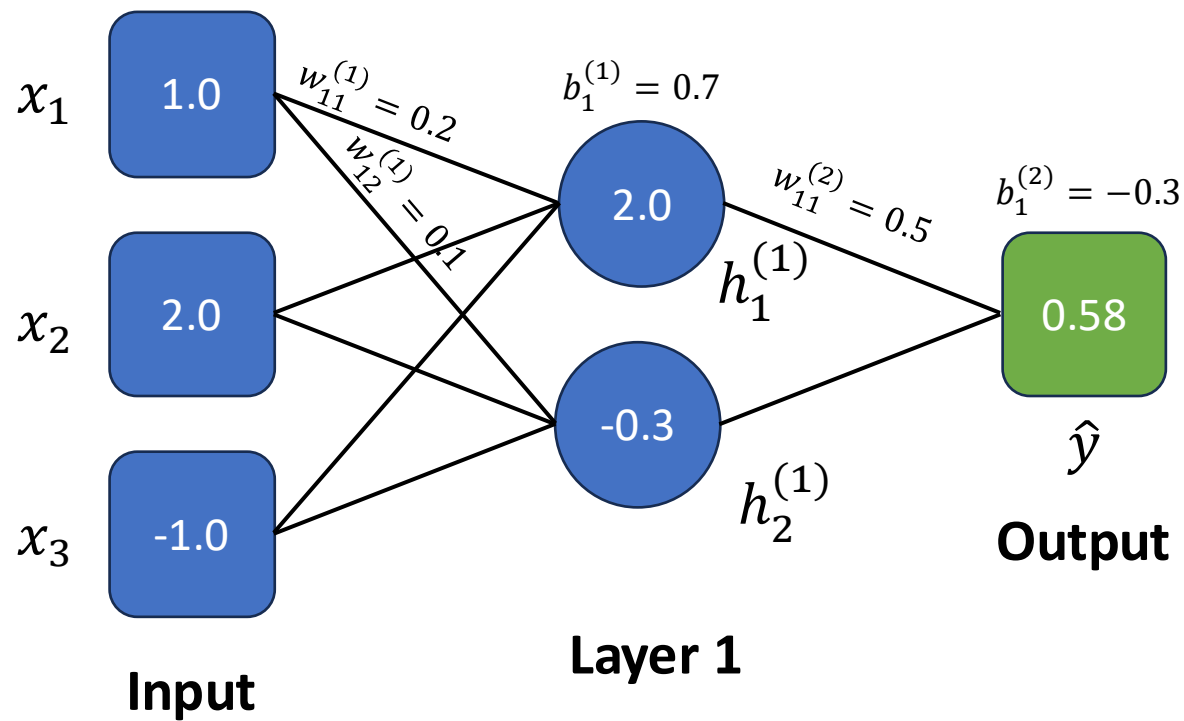
$$\frac{\partial L}{\partial w}, \quad \frac{\partial L}{\partial b}$$

## 4. Update the parameters

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

What is  $\eta$ ?

# Backpropagation Practice

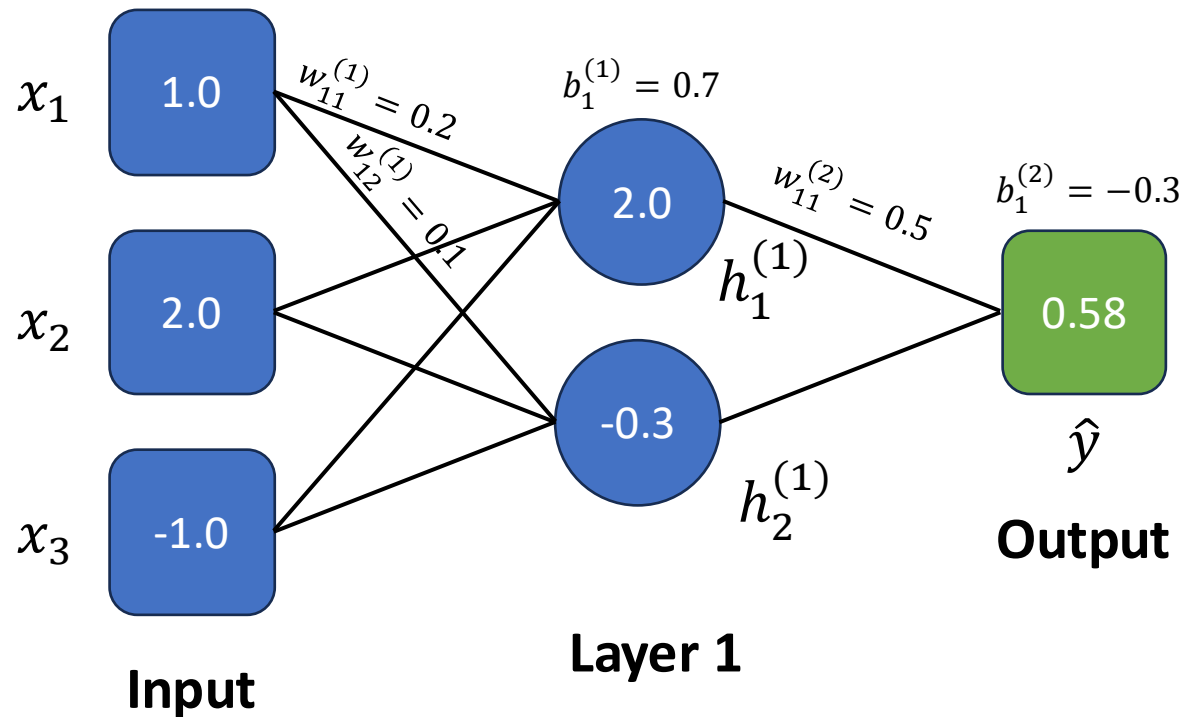


Suppose that the true value is  $y = 1.0$ , the learning rate  $\eta = 0.1$ , the loss function is MSE, let's do a backpropagation pass.

| Weight / bias  | Value |
|----------------|-------|
| $w_{11}^{(1)}$ | 0.2   |
| $w_{12}^{(1)}$ | 0.1   |
| $w_{21}^{(1)}$ | 0.3   |
| $w_{22}^{(1)}$ | 0.4   |
| $w_{31}^{(1)}$ | -0.5  |
| $w_{32}^{(1)}$ | 0.3   |
| $b_1^{(1)}$    | 0.7   |
| $b_2^{(1)}$    | -0.9  |

| Weight / bias  | Value |
|----------------|-------|
| $w_{11}^{(2)}$ | 0.5   |
| $w_{21}^{(2)}$ | 0.4   |
| $b_1^{(2)}$    | -0.3  |

# Backpropagation Practice



Negative gradient  $\rightarrow$  increasing  $\hat{y}$  would reduce the loss.

**Step 1: Calculate the MSE loss**

$$L = (y - \hat{y})^2$$

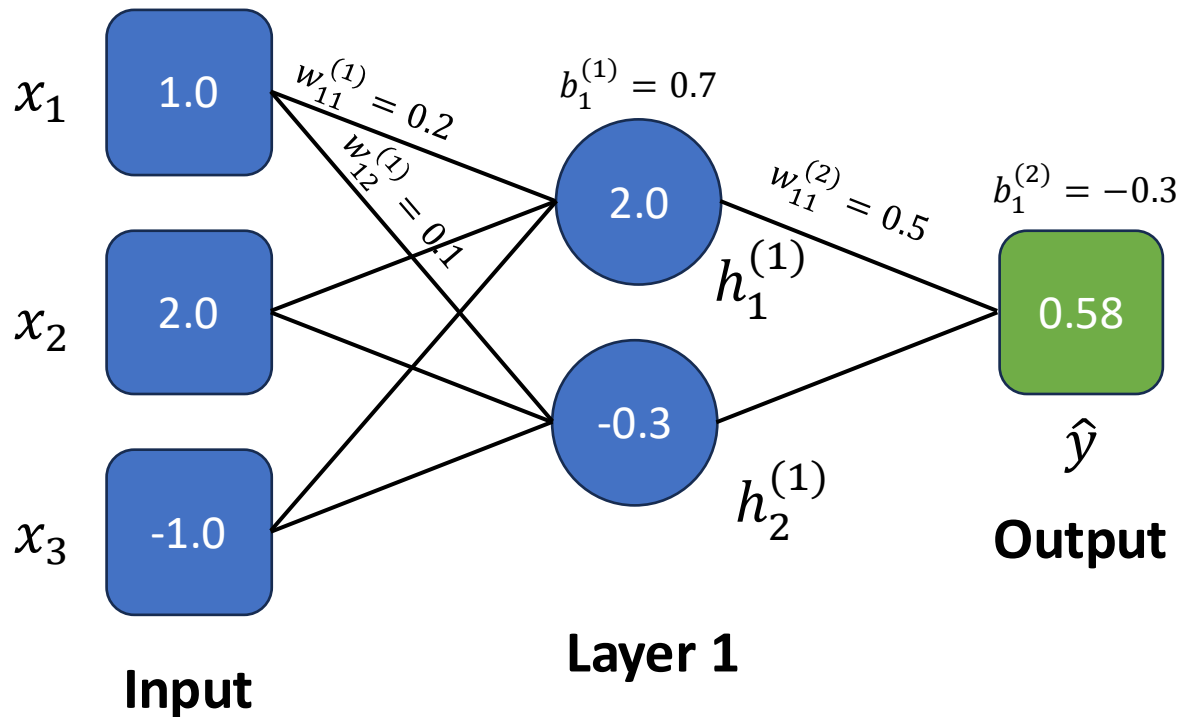
$$L = (1.0 - 0.58)^2 = \boxed{0.1764}$$

**Step 2: Backpropagation at the output**

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

$$\frac{\partial L}{\partial \hat{y}} = 2(0.58 - 1.0) = \boxed{-0.84}$$

# Backpropagation Practice



What about the second weight and the bias?

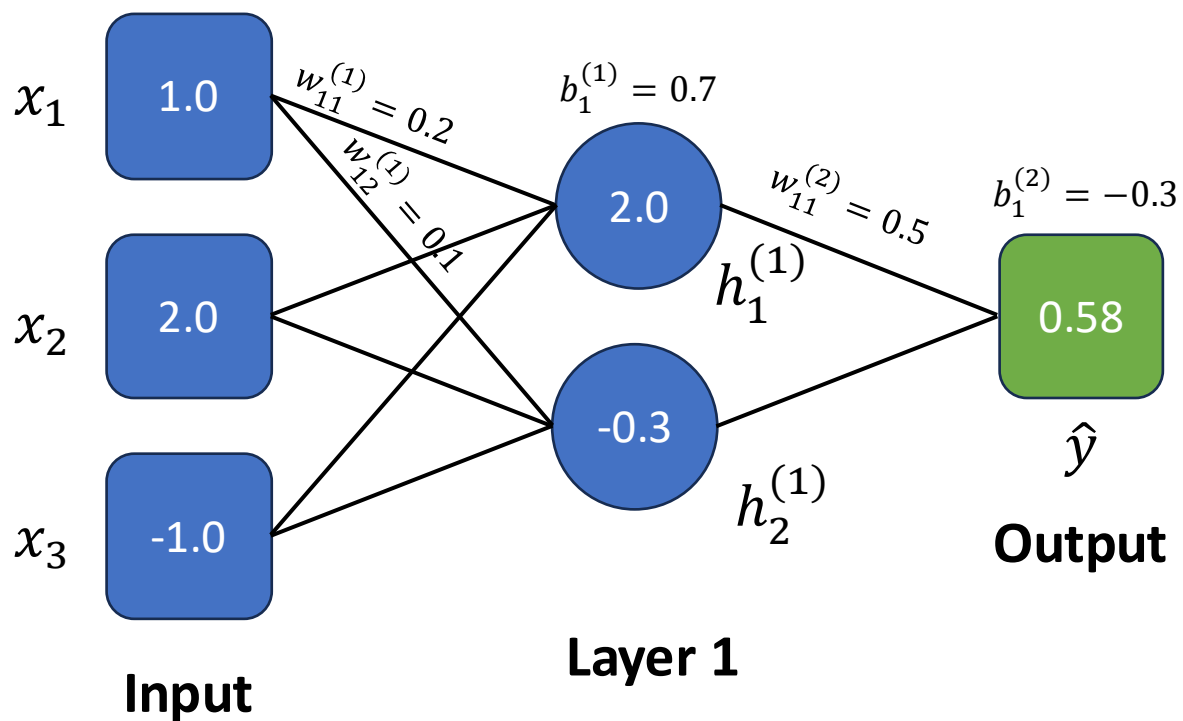
**Step 3: How does each output weight affect the loss?**

$$\hat{y} = w_{11}^{(2)} h_1^{(1)} + w_{21}^{(2)} h_2^{(1)} + b_1^{(2)}$$

Using the **chain rule**:

$$\begin{aligned} \frac{\partial L}{\partial w_{11}^{(2)}} &= \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_{11}^{(2)}} \\ &= (-0.84)(2.0) \\ &= \boxed{-1.68} \end{aligned}$$

# Backpropagation Practice



What are the updated parameters?

$$\frac{\partial L}{\partial w_{21}^{(2)}} = (-0.84)(-0.3) = \boxed{0.252}$$

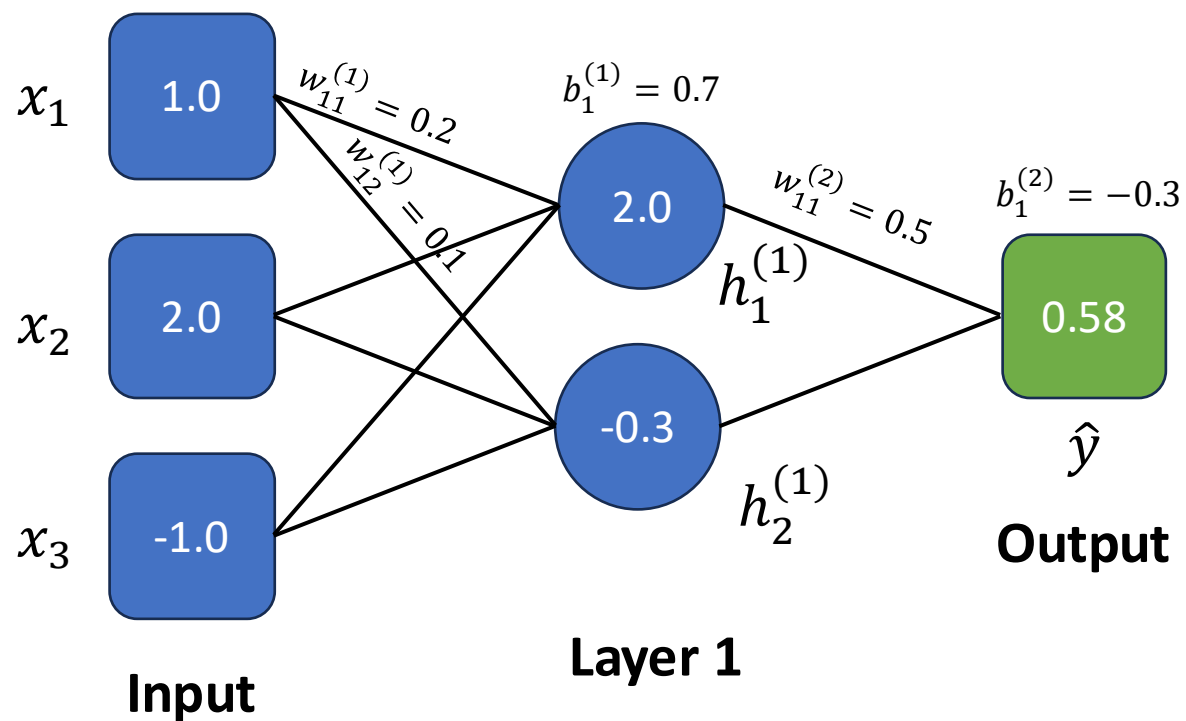
$$\frac{\partial L}{\partial b_1^{(2)}} = \boxed{-0.84}$$

Step 4: Update the parameters

$$\eta = 0.1$$

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

# Backpropagation Practice



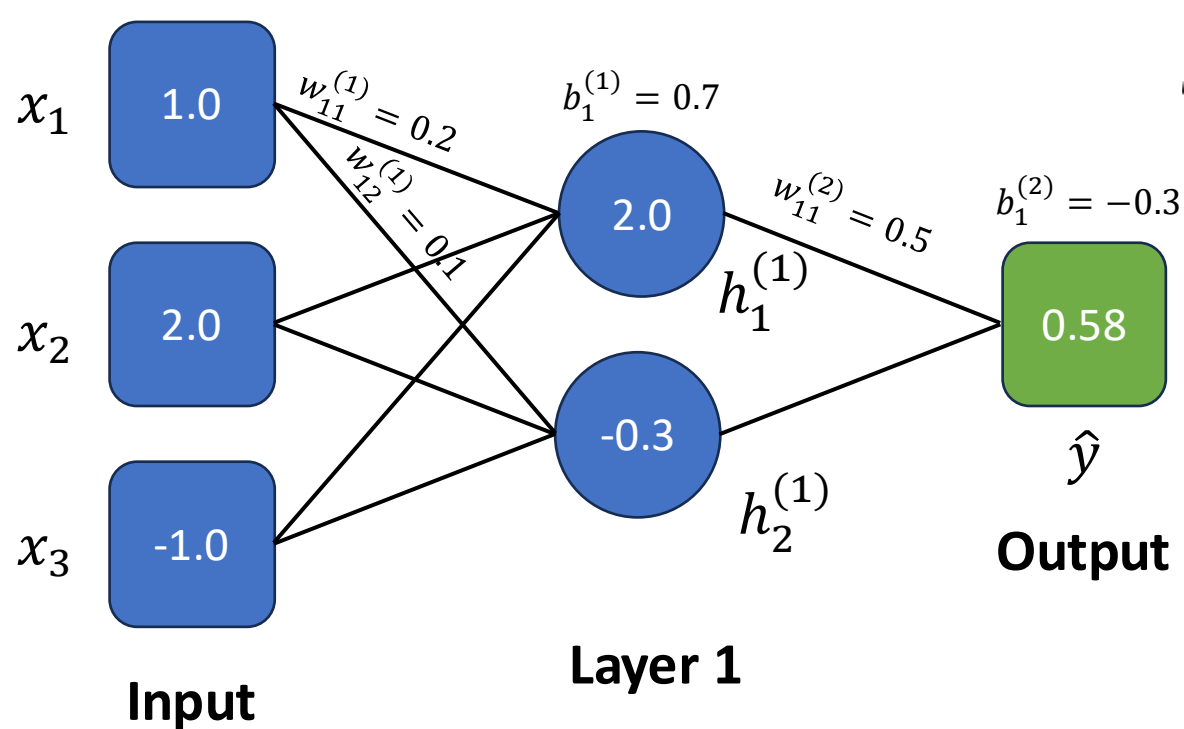
$$b_{1,\text{new}}^{(2)} = -0.3 - (0.1)(-0.84)$$
$$= \boxed{-0.216}$$

$$\eta = 0.1 \quad w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

$$w_{11,\text{new}}^{(2)} = 0.5 - (0.1)(-1.68)$$
$$= \boxed{0.668}$$

$$w_{21,\text{new}}^{(2)} = 0.4 - (0.1)(0.252)$$
$$= \boxed{0.3748}$$

# Backpropagation Practice



$$h_1^{(1)} = w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + w_{31}^{(1)} x_3 + b_1^{(1)}$$

$$h_2^{(1)} = w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 + w_{32}^{(1)} x_3 + b_2^{(1)}$$

Using the chain rule again:

$$\begin{aligned} \frac{\partial L}{\partial w_{11}^{(1)}} &= \frac{\partial L}{\partial h_1^{(1)}} \frac{\partial h_1^{(1)}}{\partial w_{11}^{(1)}} \\ &= (-0.42)(1.0) \\ &= -0.42 \end{aligned}$$

# Backpropagation Practice

$$\frac{\partial L}{\partial w_{21}^{(1)}} = (-0.42)(2.0) = -0.84$$

$$\frac{\partial L}{\partial w_{22}^{(1)}} = (-0.336)(2.0) = -0.672$$

$$\frac{\partial L}{\partial w_{31}^{(1)}} = (-0.42)(-1.0) = 0.42$$

$$\frac{\partial L}{\partial w_{32}^{(1)}} = (-0.336)(-1.0) = 0.336$$

$$\frac{\partial L}{\partial b_1^{(1)}} = -0.42$$

$$\frac{\partial L}{\partial b_2^{(1)}} = -0.336$$

$$\frac{\partial L}{\partial w_{12}^{(1)}} = (-0.336)(1.0) = -0.336$$

# Backpropagation Practice

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

$$w_{11,\text{new}}^{(1)} = 0.2 - (0.1)(-0.42) = 0.242$$

$$w_{12,\text{new}}^{(1)} = 0.1 - (0.1)(-0.336) = 0.1336$$

$$w_{21,\text{new}}^{(1)} = 0.3 - (0.1)(-0.84) = 0.384$$

$$w_{22,\text{new}}^{(1)} = 0.4 - (0.1)(-0.672) = 0.4672$$

$$w_{31,\text{new}}^{(1)} = -0.5 - (0.1)(0.42) = -0.542$$

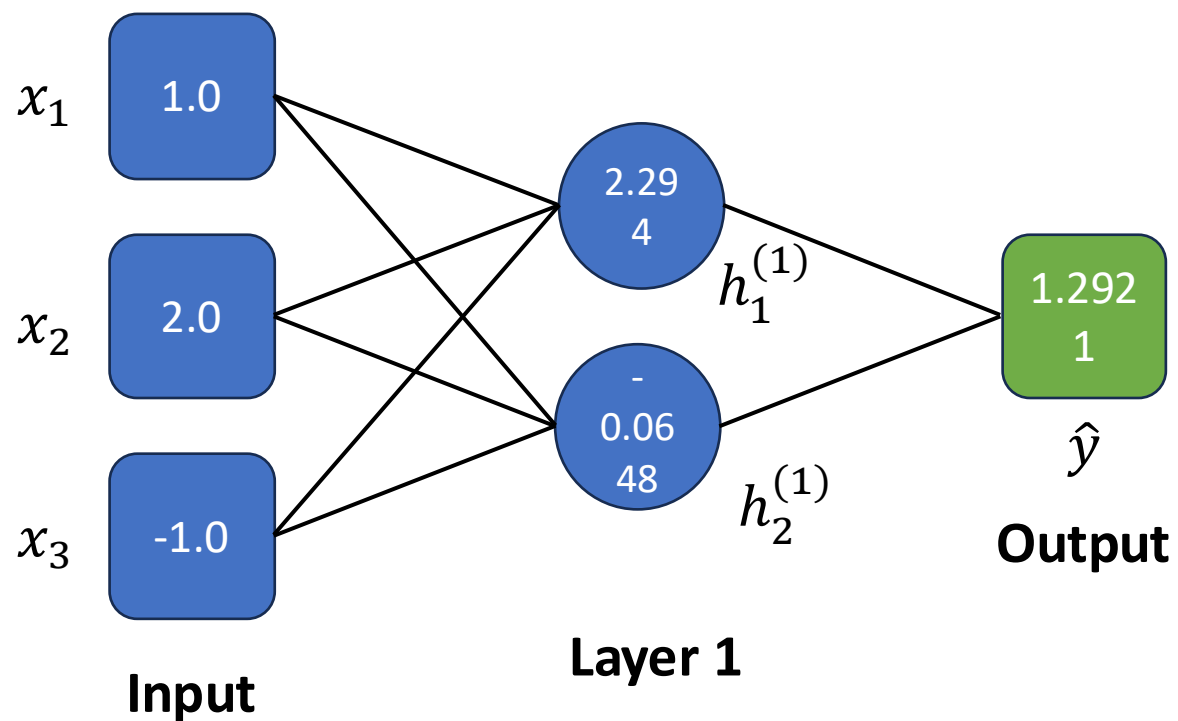
$$w_{32,\text{new}}^{(1)} = 0.3 - (0.1)(0.336) = 0.2664$$

$$b_{1,\text{new}}^{(1)} = 0.7 - (0.1)(-0.42) = 0.742$$

$$b_{2,\text{new}}^{(1)} = -0.9 - (0.1)(-0.336) = -0.8664$$

**Backpropagation sends the loss gradient backward through the network so each weight and bias can be updated.**

# Backpropagation Practice



After one update step, the prediction changes from 0.58 to 1.2921, and the loss decreases from 0.1764 to 0.0853.

| Weight / bias  | Value   |
|----------------|---------|
| $w_{11}^{(1)}$ | 0.2420  |
| $w_{12}^{(1)}$ | 0.1336  |
| $w_{21}^{(1)}$ | 0.3840  |
| $w_{22}^{(1)}$ | 0.4672  |
| $w_{31}^{(1)}$ | -0.5420 |
| $w_{32}^{(1)}$ | 0.2664  |
| $b_1^{(1)}$    | 0.7420  |
| $b_2^{(1)}$    | -0.8664 |

| Weight / bias  | Value   |
|----------------|---------|
| $w_{11}^{(2)}$ | 0.6680  |
| $w_{21}^{(2)}$ | 0.3748  |
| $b_1^{(2)}$    | -0.2160 |

# Train / Validation / Validation / Test Split

In neural networks, we typically have two types of validation data.

## Training set

- Used to update **weights and biases**
- Seen repeatedly during gradient descent

## Validation 1: Early stopping

- Checked after each epoch
- Chooses the best epoch / best weights
- Does not update weights directly

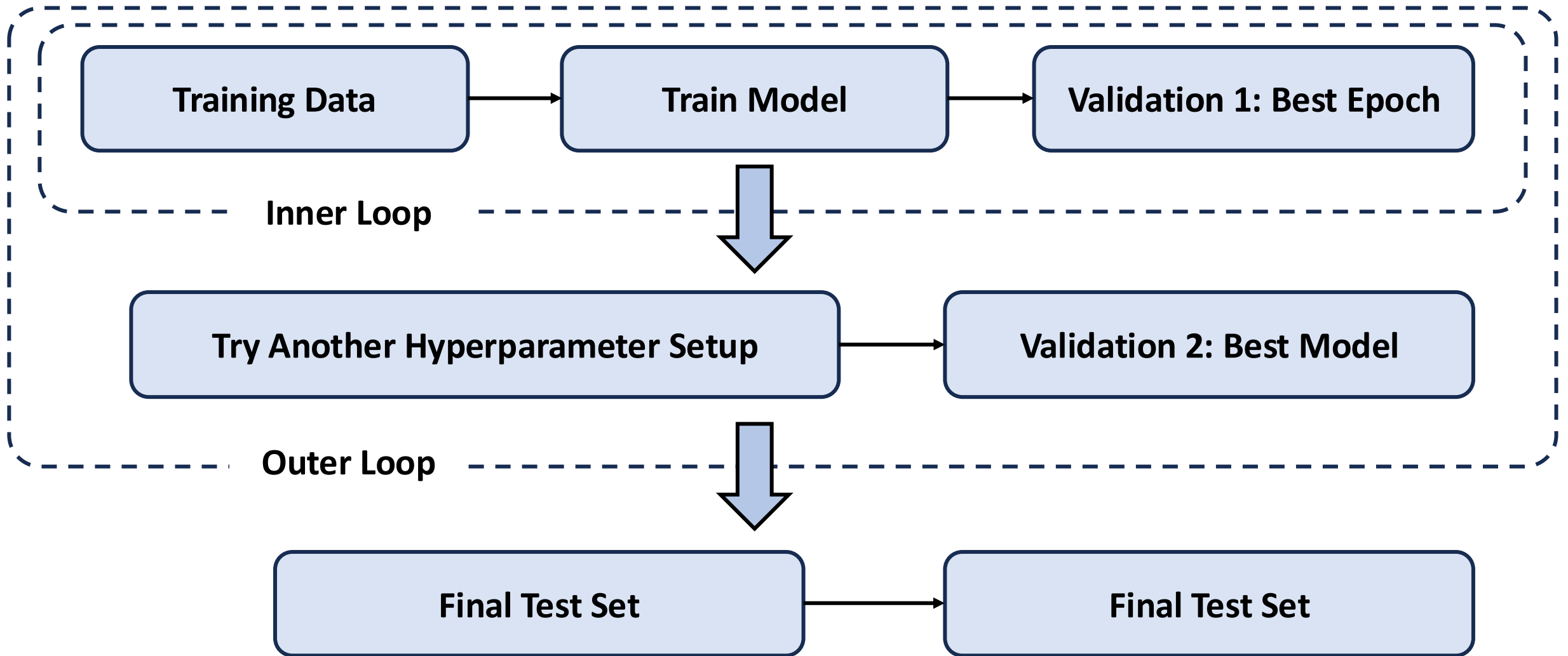
## Validation 2: Hyperparameter tuning

- Compares different model configurations
- Chooses learning rate, layers, neurons, batch size, etc.

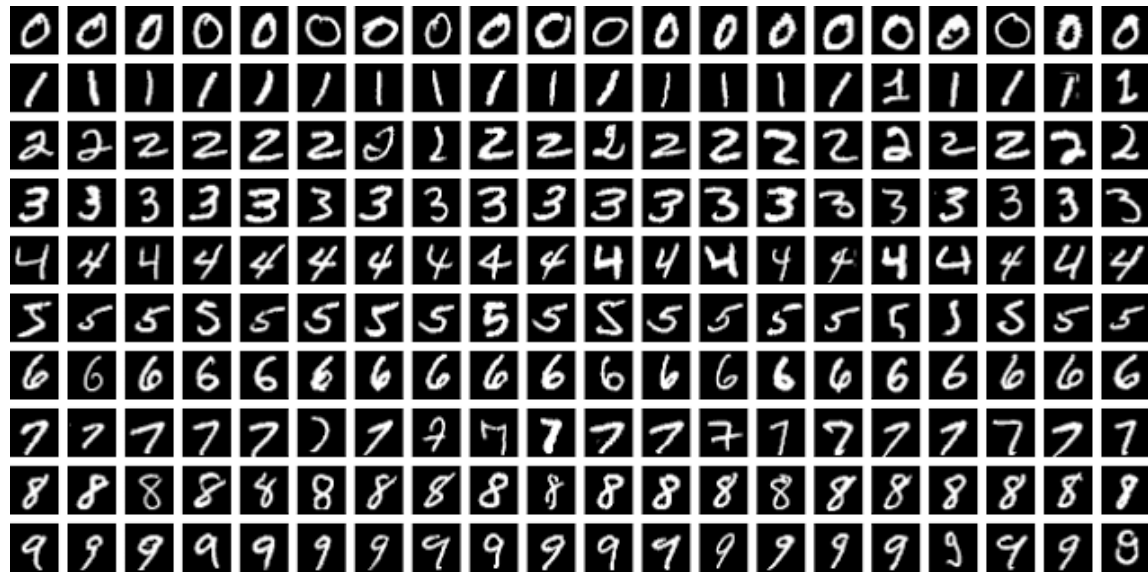
## Test Set

- Used once at the very end
- Gives the final estimate of performance on unseen data

# The Full Workflow



# MNIST Classification Example



## What is MNIST?

- 60k training images + 10k test images
- Each image is 28 x 28 grayscale pixels
- Labels are digits 0 to 9 → a 10-class classification problem

```
import numpy as np

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

tf.keras.utils.set_random_seed(26)

# 1. Load MNIST
(x, y), (x_test, y_test) = keras.datasets.mnist.load_data()

x, x_test = x / 255.0, x_test / 255.0

print(x.shape, y.shape, x_test.shape, y_test.shape)

(60000, 28, 28) (60000,) (10000, 28, 28) (10000,)
```

## Why divide by 255?

- Pixel values originally range from [0, 255], convert them to [0, 1].

# MNIST: Data Split and Model Setup

```
# Train / Validation 1 / Validation 2
idx = np.random.permutation(len(x))
x, y = x[idx], y[idx]

x_train, y_train = x[:48000], y[:48000]
x_val1, y_val1 = x[48000:54000], y[48000:54000]
x_val2, y_val2 = x[54000:], y[54000:]
```

```
# 2. Simple fully connected neural network
def build_model(n_hidden):
    model = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(n_hidden, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])

    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

## Data Split

- **Train:** update weights and biases
- **Validation 1:** select the best weights during training
- **Validation 2:** compare hyperparameter settings

## Network architecture

- Flatten:  $28 \times 28 \rightarrow 784$  inputs
- Dense( $n_{\text{hidden}}$ , ReLU): hidden layer
- Dense(10, Softmax): one probability for each digit

## Compile

- **Adam:** optimizer that updates the parameters
- **Sparse categorical cross-entropy:** loss for integer class labels 0, 1, ..., 9.
- **Accuracy:** metric used to evaluate classification

$$28 \times 28 \rightarrow 784 \rightarrow n_{\text{hidden}} \rightarrow 10$$

# MNIST: Hyperparameter Tuning and Model Selection

```
# 3. Hyperparameter tuning
```

```
models = {}
```

```
for n_hidden in [64, 128]:
```

```
    model = build_model(n_hidden)
```

```
    path = f"best_{n_hidden}.weights.h5"
```

```
    checkpoint = keras.callbacks.ModelCheckpoint(
        path, monitor="val_loss", save_best_only=True, save_weights_only=True)
```

```
    model.fit(x_train, y_train, validation_data=(x_val1, y_val1),
              epochs=20, batch_size=128, callbacks=[checkpoint], verbose=0)
```

```
    model.load_weights(path)
```

```
    models[n_hidden] = model
```

```
# Compare models using Validation 2
```

```
val_acc = {n: m.evaluate(x_val2, y_val2, verbose=0)[1]
```

```
            for n, m in models.items()]
```

```
best_hidden = max(val_acc, key=val_acc.get)
```

```
best_model = models[best_hidden]
```

What are we tuning?

- Number of hidden neurons  $\in \{64, 128\}$

For each model

- Train for up to **20 epochs**
- Use **batch size = 128**
- **Validation 1** monitors val\_loss
- ModelCheckpoint saves the weights with the **lowest validation loss**

Then compare models

- Restore the best weights for each model
- Evaluate each model on **Validation 2**
- Select the hidden-layer size with the highest validation accuracy

Train → Val 1: best weights → Val 2: best architecture

# MNIST: Final Test Evaluation

```
# 4. Test only once
```

```
test_loss, test_acc = best_model.evaluate(x_test, y_test)
```

```
print("Test accuracy:", test_acc)
```

```
313/313 ————— 0s 285us/step – accuracy: 0.9768 – loss: 0.0796
```

```
Test accuracy: 0.9768000245094299
```

```
from sklearn.metrics import ConfusionMatrixDisplay
```

```
import matplotlib.pyplot as plt
```

```
# Final test
```

```
test_loss, test_acc = best_model.evaluate(x_test, y_test, verbose=0)
```

```
# Confusion matrix
```

```
y_pred = np.argmax(best_model.predict(x_test, verbose=0), axis=1)
```

```
ConfusionMatrixDisplay.from_predictions(
```

```
    y_test, y_pred,
```

```
    cmap="Blues",
```

```
    values_format="d"
```

```
)
```

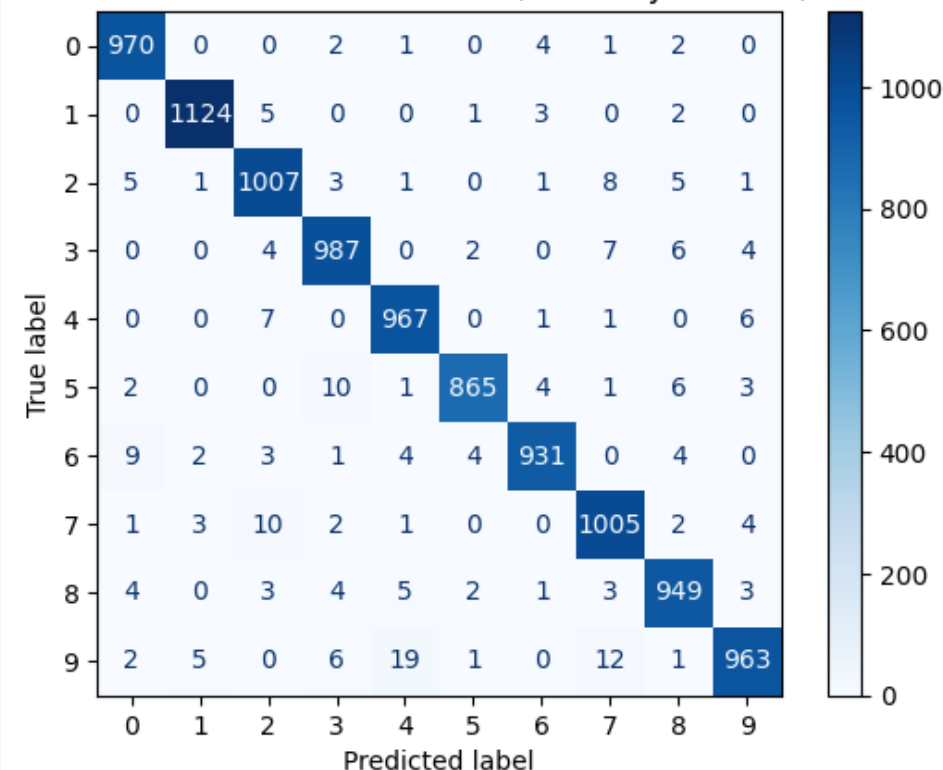
```
plt.title(f"MNIST Test Confusion Matrix (Accuracy = {test_acc:.3f})")
```

```
plt.show()
```

## Final test

- Evaluate the **selected model only once**
- Report final **test loss** and **test accuracy**
- Here: **accuracy = 97.7%**

MNIST Test Confusion Matrix (Accuracy = 0.977)



**What does backpropagation actually do?**

- A. Directly chooses the best network architecture
- B. Calculates how the loss changes with each weight and bias
- C. Splits the dataset into training and test sets
- D. Converts inputs into probabilities

Answer: B.

**For MNIST digit classification with 10 classes, which combination makes sense??**

- A. One output + MSE
- B. 10 outputs with softmax + categorical cross-entropy
- C. 10 outputs with ReLU + MAE
- D. One output with sigmoid

Answer: B.

## In this lecture, we learned

- *How neural networks are structured: neurons, weights, biases, hidden layers, and activation functions*
- *How neural networks learn: forward propagation → loss → backpropagation → parameter updates*

## Next lecture:

- *How neural networks are trained in practice*
  - *Epochs, batches, and learning rate*
  - *Optimizers such as SGD and Adam*
  - *Overfitting, training curves, and regularization*